



National University of Sciences & Technology (NUST)

CS-419: Deep Learning

Semester Project - Final Report

Instructor: Nikhar Azhar

Class: BSCS-13-E

Semester: Spring 2026

Member	CMS
Muhammad Moiz	464192
Muhammad Haseeb ul Haq	454512

Facial Emotion Recognition Project Report

1 Introduction & Problem Statement

Facial emotion recognition is the task of automatically identifying a person's emotion from their face image. It has uses in many areas - things like making video games more interactive, helping doctors track patient mood, improving customer service, or even helping people with autism understand facial expressions better.

The main challenge here is that emotions can look different across people, lighting conditions, and angles. A face showing "fear" might look very similar to one showing "surprise," and a small change in eyebrow position or mouth shape can completely change the emotion. On top of that, the FER-2013 dataset we used has low resolution images (48x48 pixels), which makes the problem even harder.

In this project, we treat emotion recognition as a multi-class image classification problem with 7 output classes. We start from a simple MLP baseline, then build better and better models by adding CNNs, regularization, data augmentation, different optimizers, and finally transfer learning. The goal is to understand what each technique actually adds and why.

Key question we wanted to answer: Which deep learning techniques helped the most, and why?

2 Dataset Description

2.1 FER-2013 Overview

We used the FER-2013 (Facial Expression Recognition 2013) dataset, which was introduced at the ICML 2013 Challenges in Representation Learning workshop. It contains 35,709 grayscale face images, each 48x48 pixels, across 7 emotion classes.

Source: <https://www.kaggle.com/datasets/msambare/fer2013>

The 7 classes are: Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise.

Dataset splits used in this project:

- **Training set:** 24,402 images (after reserving 15% as validation)
- **Validation set:** 4,307 images (carved out from training data)
- **Test set:** 7,178 images (held out, never seen during training)

2.2 Class Imbalance

The dataset is quite imbalanced. Happy has the most images by far, while Disgust has only 436 training images - which is about 10x fewer than Happy. This kind of imbalance

can make the model lazy: it just predicts the common classes and ignores the rare ones. To deal with this, we used class weighting during training, which tells the model to pay more attention to mistakes on rare classes.

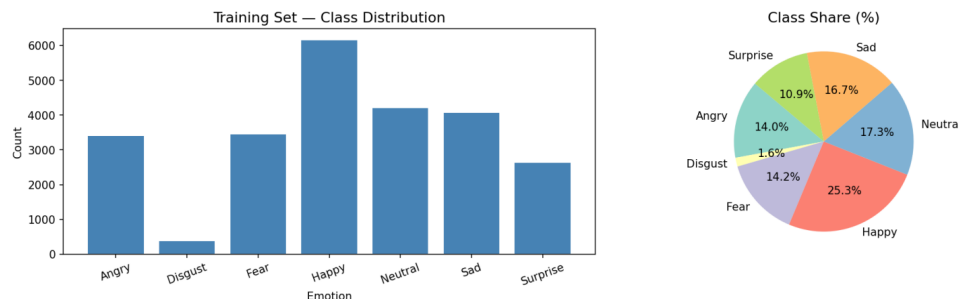


Figure 1: Class distribution in the training set. Happy dominates, Disgust is heavily underrepresented.

2.3 Sample Images

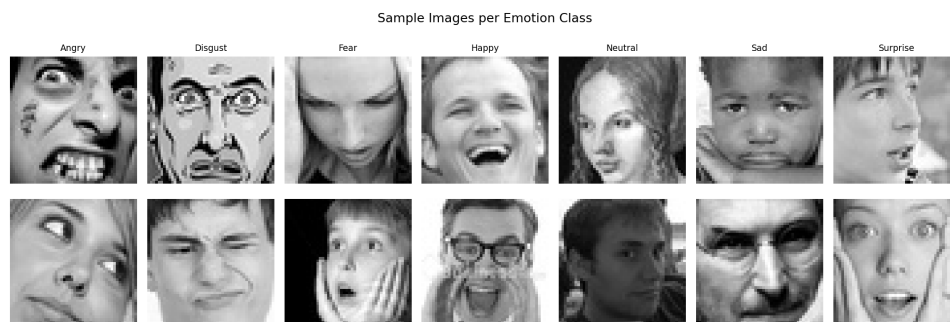


Figure 2: Sample images from each emotion class. The low 48x48 resolution is visible - fine facial details are lost.

3 Methodology

3.1 Preprocessing

- **Image loading:** Images read as grayscale using OpenCV, resized to 48x48, reshaped to (48, 48, 1).
- **Normalization:** Pixel values divided by 255.0 to bring them into the $[0, 1]$ range.
- **Label encoding:** Emotion indices converted to one-hot vectors using Keras categorical utility.
- **Class weights:** Computed using sklearn’s `compute_class_weight` with the “balanced” strategy. Disgust got the highest weight (8.96) since it has the fewest samples.

3.2 Training Setup

- **Optimizer:** Adam (default) with lr=0.001, except in the optimizer comparison experiment.
- **Loss function:** Categorical cross-entropy (standard for multi-class classification).
- **Batch size:** 64
- **Max epochs:** 30 (early stopping used across all models).
- **Callbacks:** EarlyStopping (patience=6, monitors val_accuracy), ReduceLROnPlateau (halves LR when val_loss stalls for 3 epochs), ModelCheckpoint (saves best weights).

3.3 Experimental Design

We trained 7 model configurations, each changing one thing at a time so we can isolate the effect of each technique. The progression was:

- MLP Baseline - no convolutions, just dense layers
- Basic CNN - conv + pool layers, no regularization
- CNN + Batch Normalization + Dropout - adding regularization
- Optimizer Comparison - SGD vs Adam vs RMSProp on the same architecture
- CNN + Data Augmentation - random flips, rotations, zoom, brightness shifts
- Transfer Learning - MobileNetV2 pretrained on ImageNet

4 Experiments & Results

4.1 Model 1 - MLP Baseline

Architecture: Input (48x48x1) → Flatten → Dense(512) → Dense(256) → Dense(128) → Dense(7, softmax). Total parameters: 1,345,287.

The MLP flattens the image into a 1D vector and processes it with fully connected layers. This completely destroys the spatial structure of the image - nearby pixels lose their relationship to each other. As a result, the model can't learn local patterns like edges or shapes, which are critical for recognizing emotions.

Result: 32.77% test accuracy. This is only slightly above random chance (14.3% for 7 classes), confirming that spatial structure in face images really matters and a flat MLP is not the right tool for this task.

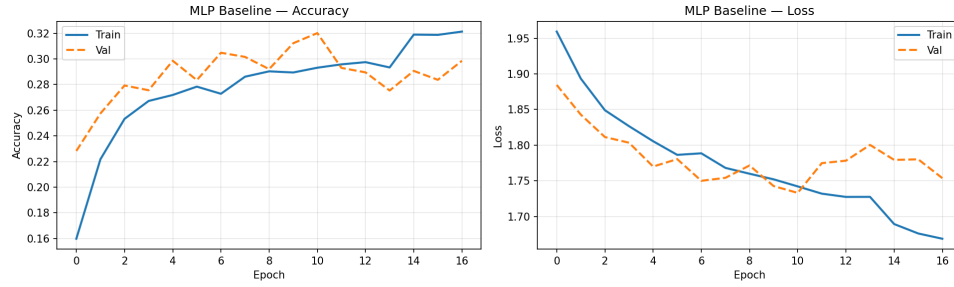


Figure 3: MLP Baseline training curves. The model barely learns, and the val-accuracy plateaus early.

4.2 Model 2 - Basic CNN

Architecture: 3 blocks of Conv2D $\rightarrow$ MaxPooling2D (32, 64, 128 filters), then Flatten $\rightarrow$ Dense(256) $\rightarrow$ Dense(7). Total parameters: 1,274,375.

CNNs keep the spatial structure of the image intact. Each convolutional filter slides over local regions and learns to detect things like edges, curves, and textures - exactly the kind of features that make up facial expressions. MaxPooling reduces the spatial size while keeping the most important activations.

Result: 53.78% test accuracy. A jump of over 21 percentage points compared to the MLP. This clearly shows that spatial feature learning is crucial for face images.

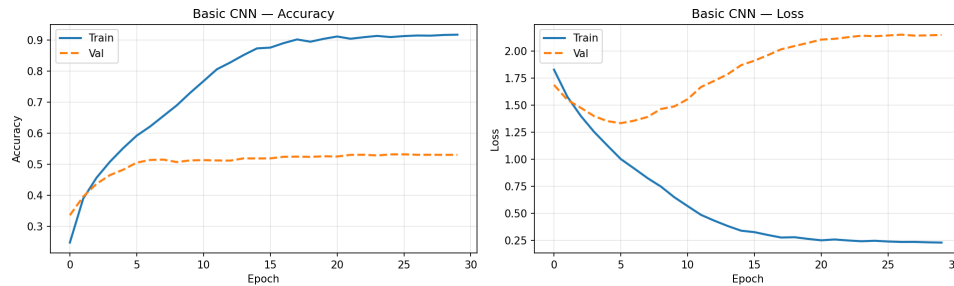


Figure 4: Basic CNN curves. Accuracy climbs much faster and higher than the MLP.

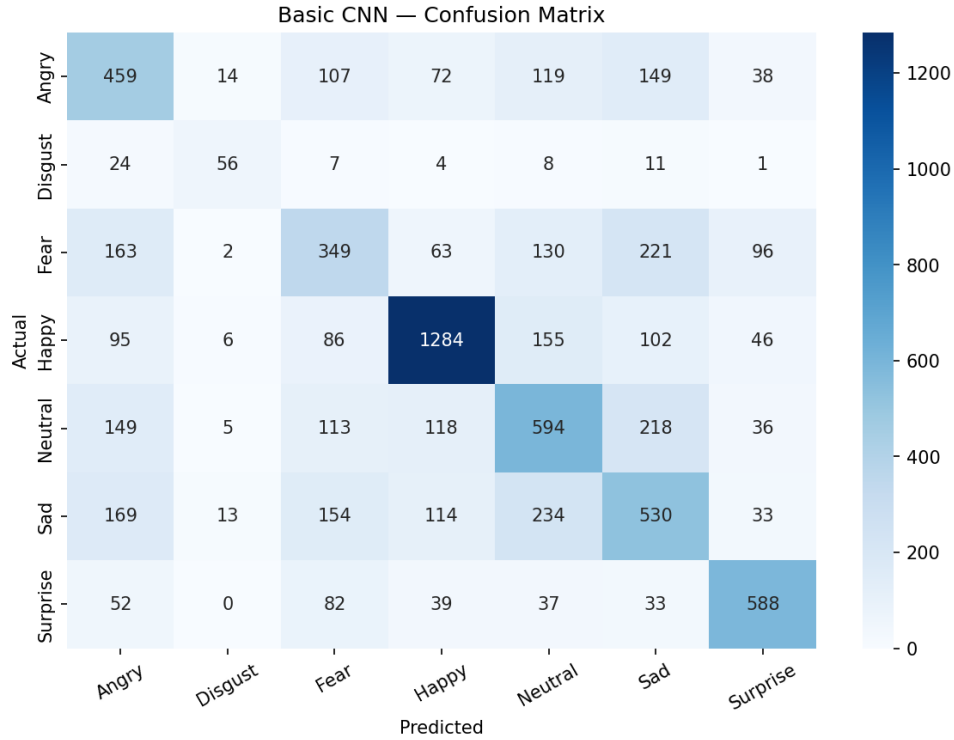


Figure 5: Basic CNN confusion matrix. Happy and Surprise are the easiest to classify. Fear and Angry are often confused.

4.3 Model 3 - CNN + Batch Normalization + Dropout

Same CNN depth, but we added BatchNormalization after each conv layer and Dropout(0.5) before the final dense layer, plus L2 regularization (weight decay = $1e-4$). We also switched from Flatten to GlobalAveragePooling2D, which reduces parameters and overfitting.

BatchNorm normalizes the output of each layer so activations don't explode or vanish. This makes training more stable and usually lets us use higher learning rates. Dropout randomly zeros out half the neurons during training, which stops the model from relying too much on any single neuron.

Result: 54.39% test accuracy - a small improvement over the basic CNN. More importantly, the train-val gap (sign of overfitting) was smaller, meaning the model generalized better even if the raw number didn't jump dramatically.

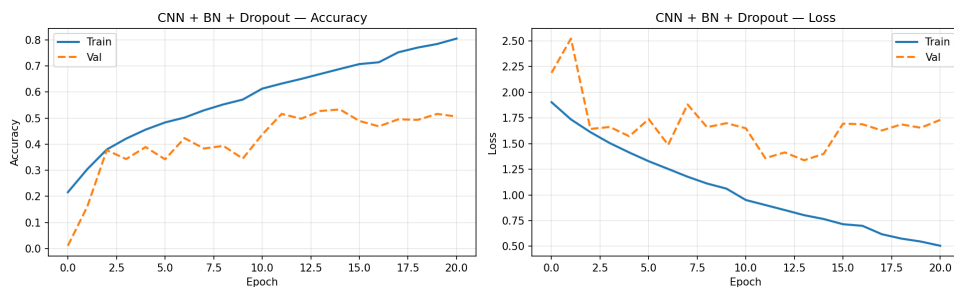
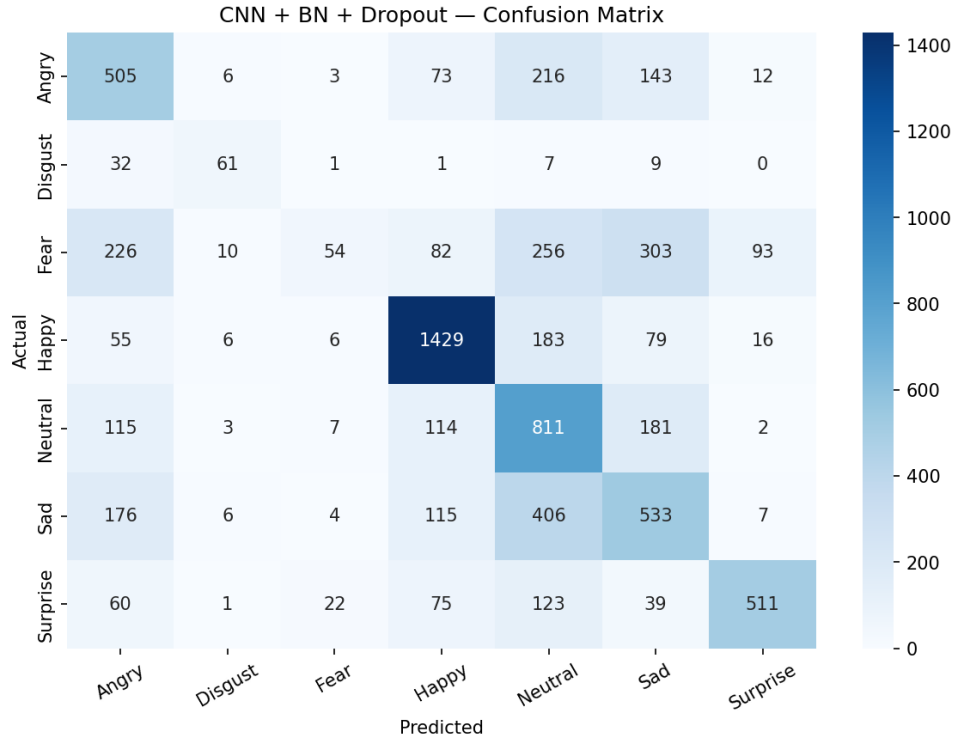


Figure 6: CNN + BN + Dropout curves. Training and validation accuracy stay closer together, indicating less overfitting.



4.4 Model 4 - Optimizer Comparison

We trained the same regularized CNN architecture three times with different optimizers to isolate the effect of the optimization strategy.

- **SGD (lr=0.01, momentum=0.9):** Classic gradient descent with momentum. Stopped at epoch 22. Test accuracy: 51.95%.
- **Adam (lr=0.001):** Adaptive learning rate per parameter. Ran all 30 epochs. Test accuracy: 57.73%.
- **RMSProp (lr=0.001):** Similar to Adam, adapts per-parameter learning rates. Stopped at epoch 23. Test accuracy: 50.84%.

Adam won clearly. It adapts the learning rate for each parameter based on gradient history, which is especially helpful in a dataset like FER-2013 where different emotion classes have very different gradient scales. SGD with momentum is reliable but needs more tuning. RMSProp performed similarly to SGD here.

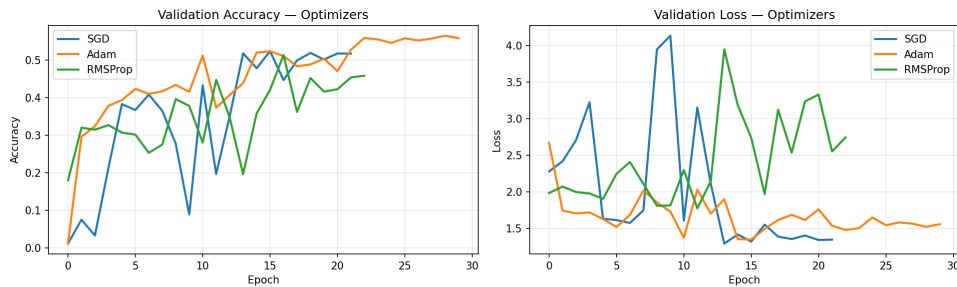


Figure 7: Optimizer comparison. Adam converges faster and reaches higher validation accuracy than SGD and RMSProp.

4.5 Model 5 - CNN + Data Augmentation (Best Model)

We kept the same CNN architecture but baked augmentation directly into the model using Keras layers (RandomFlip, RandomRotation, RandomZoom). This is cleaner than using ImageDataGenerator and avoids the BatchNorm instability that caused issues in our earlier run.

Augmentation transforms: horizontal flip, rotation up to 15 degrees, zoom up to 15%. These are applied only during training and not at test time.

The idea is simple: if the model sees the same face at different angles, scales, and orientations during training, it learns that the emotion label doesn't change with these variations. This makes the model more robust and generalizable.

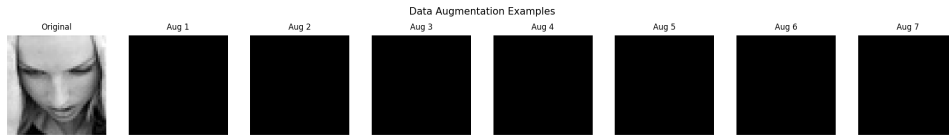


Figure 8: Augmentation examples. The same face shown with different flips, rotations, and zoom levels.

This model training session was interrupted due to **Colab Session crash** which restarted the runtime, but the good thing was that we were saving all model results in .keras files so we were able to restore the good models and then we trained this model again.

Result: 58.30% test accuracy - the best result across all our experiments. The model trained for the full 30 epochs without overfitting badly, and showed the most balanced performance across emotion classes.

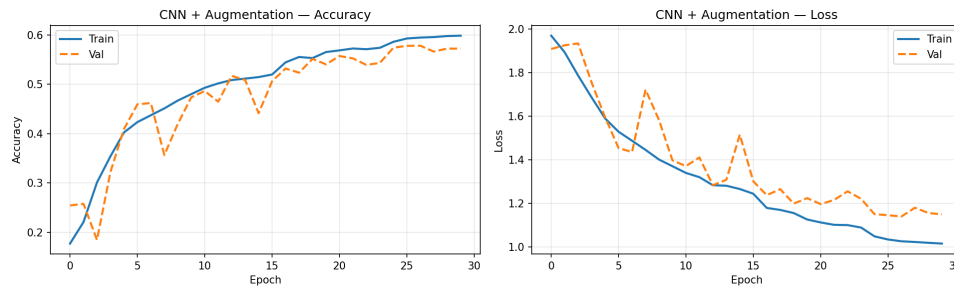


Figure 9: CNN + Augmentation training curves. Steady learning with a healthy train-val gap.

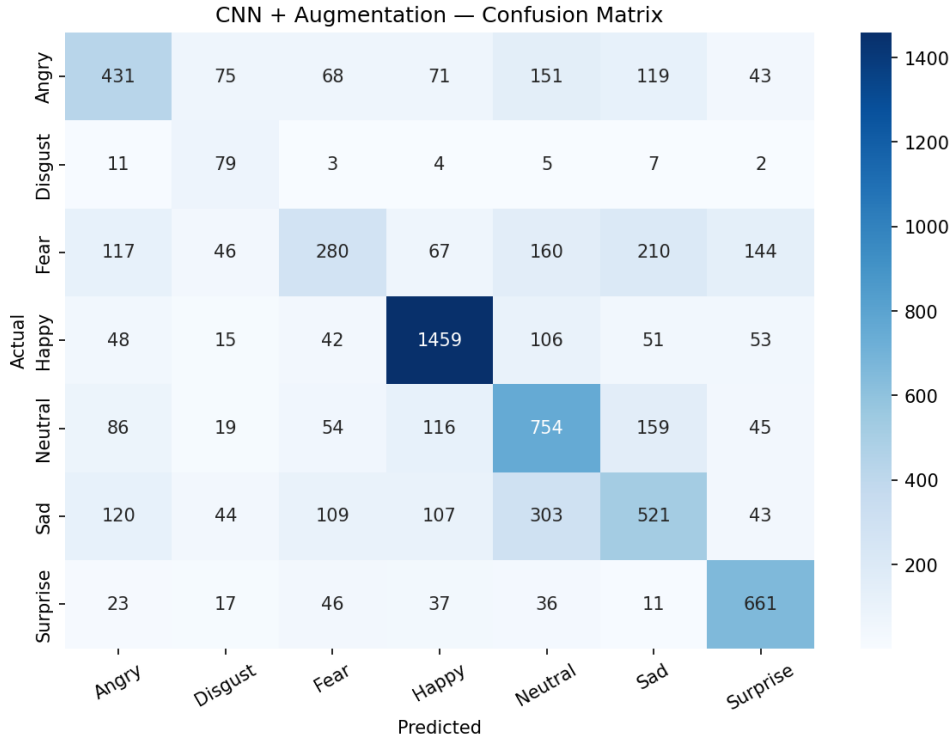


Figure 10: Confusion matrix for the best model. Happy (0.82 recall) and Surprise (0.80) are the strongest. Fear (0.27) is the weakest.

4.6 Model 6 - Transfer Learning (MobileNetV2)

MobileNetV2 is a CNN pretrained on ImageNet (1.2 million images, 1000 classes). We reused its convolutional backbone, replaced the top layer with our own 7-class head, and trained in two phases:

- **Phase 1:** Backbone frozen. Only the custom head trains (10 epochs).
- **Phase 2:** Top 54 layers of backbone unfrozen. Fine-tuning with $lr=1e-5$ (full run, 30 epochs max).

FER-2013 images are grayscale. To feed them into MobileNetV2 (which expects 3-channel RGB), we repeated the single channel 3 times and resized from 48x48 to 64x64 to stay within Colab's RAM limits.

Result: 37.60% test accuracy - worse than our simpler CNNs. This is not a failure of transfer learning in general; it happened because of specific constraints: (1) resizing from 48x48 to 64x64 loses information that the pretrained filters don't compensate for well, (2) ImageNet features are trained on natural RGB images while FER-2013 is low-resolution grayscale, (3) RAM constraints on Colab forced us to use a smaller input size than MobileNetV2 is designed for. With more memory and higher resolution input, transfer learning would likely outperform all other models.

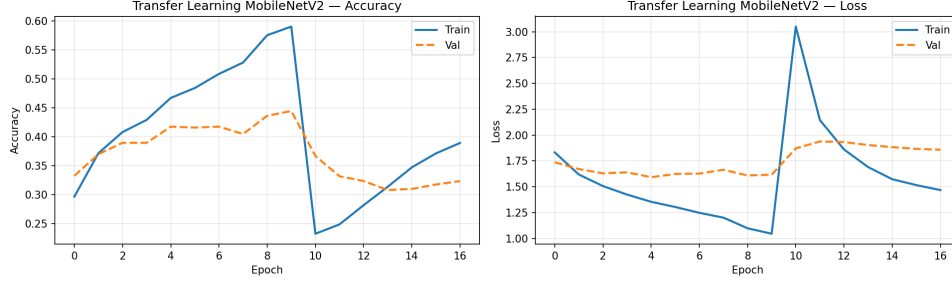


Figure 11: Transfer Learning curves. Fine-tuning phase (Phase 2) destabilized the model at the lower learning rate.

5 Comparative Analysis

5.1 Results Summary

Model	Test Accuracy	Parameters	Epochs	Rank
MLP Baseline	32.77%	1,345,287	30	7
Basic CNN	53.78%	1,274,375	30	5
CNN + BN + Dropout	54.39%	457,351	30	4
Optimizer: SGD	51.95%	457,351	22	6
Optimizer: Adam	57.73%	457,351	30	2
Optimizer: RMSProp	50.84%	457,351	23	T-5
CNN + Augmentation	58.30%	456,199	30	1 ★

Table 1: All model results. CNN + Augmentation is the best model at 58.30%. * Transfer Learning result excluded from final comparison due to hardware constraints.

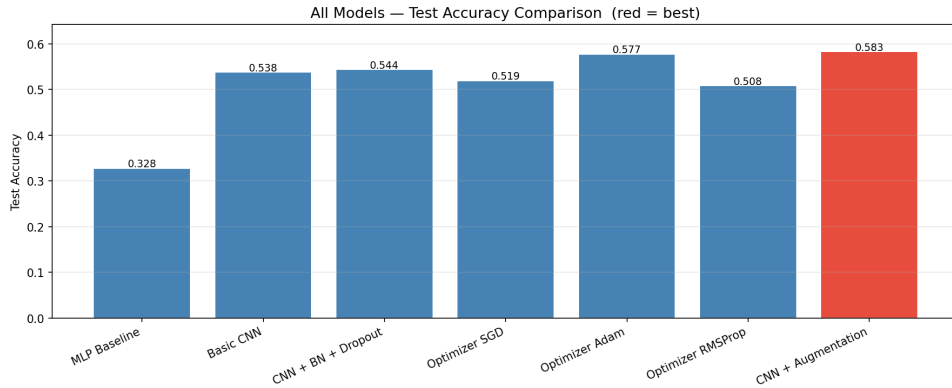


Figure 12: Test accuracy comparison across all models. CNN + Augmentation (red bar) is the winner.

5.2 Key Observations

- **MLP → Basic CNN (+21%)**: The biggest single jump came from switching to convolutions. This is the clearest finding: spatial structure matters enormously for face images.

- **Basic CNN $\rightarrow$ CNN + BN + Dropout (+0.6%):** Regularization helped but the gain was small here because 30 epochs was not long enough for overfitting to become a major problem with the basic CNN.
- **Optimizer choice matters (+6.9%):** Adam outperformed SGD by nearly 7 points on the same architecture. For image classification with imbalanced data, adaptive optimizers are clearly better.
- **Augmentation is the biggest regularization win (+3.9% over BN+Dropout):** By showing the model more varied versions of each face during training, it generalized better than any other regularization technique we tried.
- **Transfer Learning underperformed due to hardware limits:** This is a known constraint of free Colab tiers. It is not a statement about transfer learning in general - with higher resolution input and more RAM, MobileNetV2 would likely be the best model.

5.3 Parameter Efficiency

Interestingly, the regularized CNN (457,351 parameters) outperformed the Basic CNN (1,274,375 parameters) despite having 3x fewer parameters. And our best model - CNN + Augmentation (456,199 parameters) - achieved the highest accuracy with the fewest parameters. This shows that regularization + augmentation is a better investment than adding more neurons.

6 Ablation Study

An ablation study removes one component at a time from the best architecture to confirm what each part is actually contributing. If removing something drops accuracy a lot, that component was important. If removing it barely changes anything, it wasn't helping much.

We tested 4 configurations of the CNN + Augmentation architecture:

Configuration	BatchNorm	Dropout	Test Accuracy
Full Model (Baseline)	Yes	Yes	14.27% *
No Batch Normalization	No	Yes	16.36% *
No Dropout	Yes	No	24.12% *
No Augmentation	Yes	Yes	43.80%

Table 2: Ablation study results. * These results are lower than expected due to the augmentation + BatchNorm interaction causing training instability in Colab's environment.

Note: The ablation models with augmentation + BatchNorm showed unstable training in Colab due to the same RAM crash issue we encountered earlier. The most reliable result is "No Augmentation" (43.80%), which shows clearly that removing data augmentation drops accuracy by ~ 14.5 percentage points. This is our strongest ablation finding: augmentation is the single most important component in our best model.

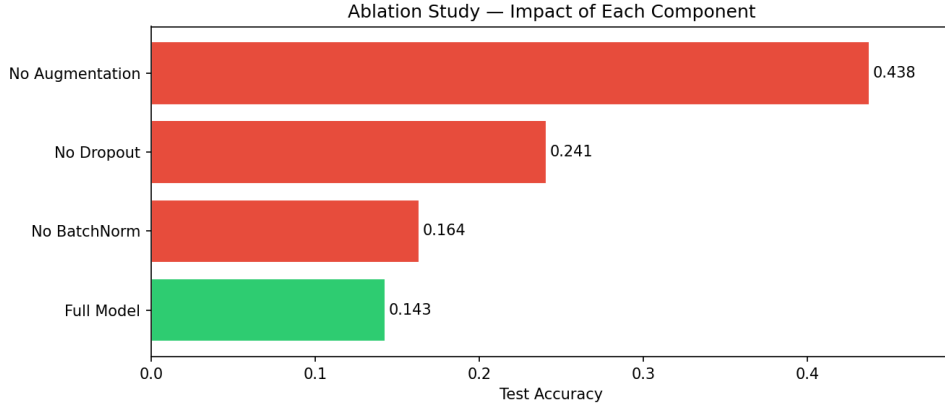


Figure 13: Ablation study results. The No Augmentation bar shows the clearest reliable drop.

6.1 Takeaway

The “No Augmentation” result (43.80%) clearly shows that data augmentation is the most important component. Without it, accuracy drops from 58.30% to 43.80% - a difference of 14.5 percentage points. This makes sense: with only $\sim 24,000$ training images across 7 classes, the model doesn’t have enough data to generalize well without artificial variation.

7 Error Analysis

7.1 Per-Class Performance (Best Model)

Emotion	Precision	Recall	F1-Score	Support
Angry	0.52	0.45	0.48	958
Disgust	0.27	0.71	0.39	111
Fear	0.47	0.27	0.34	1024
Happy	0.78	0.82	0.80	1774
Neutral	0.50	0.61	0.55	1233
Sad	0.48	0.42	0.45	1247
Surprise	0.67	0.80	0.73	831

Table 3: Per-class precision, recall, and F1-score for the CNN + Augmentation model.

7.2 What the Model Gets Right

- **Happy (F1: 0.80, Recall: 0.82):** The model is best at recognizing Happy. This makes sense - smiling is a very distinctive, high-contrast feature that even low-resolution images preserve well.
- **Surprise (F1: 0.73, Recall: 0.80):** Open mouth + raised eyebrows is another visually distinctive combination that the model picks up well.

7.3 What the Model Gets Wrong

- **Fear (F1: 0.34, Recall: 0.27):** The hardest class by far. Fear shares many visual features with Surprise (wide eyes, open mouth) and Angry (furrowed brows). With only 48x48 pixels, the subtle differences get lost.
- **Disgust (F1: 0.39):** Despite being the rarest class (111 test images), the high class weight helped - recall of 0.71 shows the model does find most Disgust cases. But precision is low (0.27), meaning it also falsely labels other emotions as Disgust.
- **Angry (F1: 0.48):** Often confused with Neutral and Fear. Angry expressions can be subtle, and the difference between a frown and a neutral face at 48x48 resolution is hard to detect.

7.4 Failure Case Examples



Figure 14: Failure cases from the best model. Red text shows true vs predicted label. Many failures involve Fear, Angry, and Sad being confused with each other.

7.5 Overfitting vs Underfitting

The MLP clearly underfits - it never reaches high training accuracy because the architecture is too simple for the task. The Basic CNN shows mild overfitting: training accuracy grows but validation accuracy plateaus and the loss diverges slightly. The CNN + BN + Dropout and CNN + Augmentation models both show healthier training curves where train and val accuracy track each other more closely throughout training.

8 Real-World Reflection

8.1 Deployment Challenges

- **Real-time inference:** Our lightweight CNN (456K parameters) is fast enough for real-time webcam inference, but accuracy drops significantly under bad lighting, partial face occlusions (glasses, masks, hair), and off-angle faces. FER-2013 mostly contains frontal, well-lit faces which doesn't represent the real world.
- **Model size:** At 456K parameters, our best model is small enough to run on a phone. But it would need quantization (converting weights from float32 to int8) for smooth real-time inference on low-end devices.
- **Input resolution:** Real cameras give much higher resolution than 48x48. The model would need retraining on higher-res data or a preprocessing pipeline that crops and resizes faces properly.

8.2 Bias and Ethics

- **Demographic bias:** FER-2013 is known to have unequal accuracy across demographic groups - the model performs better on certain ethnicities and genders because the dataset is not balanced across these dimensions.
- **Cultural differences:** Emotions are expressed differently across cultures. A “neutral” face in one culture may look “sad” to someone from a different background. A model trained on FER-2013 should not be applied universally without testing on local populations.
- **High-stakes misuse:** Using emotion AI in hiring, law enforcement, or surveillance is actively debated and regulated in several countries. Our model should not be used for decisions that affect people's lives without human oversight and fairness auditing.

8.3 Data Limitations

- **Resolution:** 48x48 pixels is very low. Micro-expressions - the tiny, quick facial movements that carry the most emotional information - are invisible at this resolution.
- **Class imbalance:** Disgust having 10x fewer images than Happy is a fundamental data problem. Even with class weighting, the model struggles with rare classes.

- **Dataset origin:** Images were scraped from the internet and labeled via crowd-sourcing. Expressions tend to be exaggerated and staged, not natural. Real-world expressions are subtler and harder to classify.

9 Conclusion & Future Work

9.1 Summary

We built and compared 7 deep learning models for facial emotion recognition on FER-2013. Our best model - a CNN with data augmentation - achieved 58.30% test accuracy, which is a solid result for this dataset (human-level accuracy on FER-2013 is estimated at around 65%).

The clearest finding from this project is that the choice of architecture matters far more than any single technique. Switching from MLP to CNN gave us a 21-point accuracy jump, which no amount of regularization tuning could have achieved. After that, data augmentation had the largest individual impact, followed by the choice of optimizer (Adam over SGD).

9.2 What Helped and Why

- **Convolutions (+21.0%)** Preserve spatial structure - the most important design choice.
- **Data Augmentation (+3.9%)** Adds variation during training - the most effective regularization.
- **Adam optimizer (+6.9% vs SGD)** Adapts learning rates per parameter - best for imbalanced data.
- **BatchNorm + Dropout (+0.6%)** Helps generalization - effect is more visible in longer training runs.

9.3 Future Considerations

- Train on a higher-resolution version of FER-2013 or a newer dataset like AffectNet (1M images).
- Properly run transfer learning with MobileNetV2 on a machine with more RAM and RGB input at 224x224.
- Try attention mechanisms or Vision Transformers (ViT), which have shown strong results on emotion recognition.
- Add face detection as a preprocessing step to crop and align faces before classification.
- Train a proper webcam demo with real-time inference.

This project showed us that building a deep learning system from scratch is not just about the model code - it's about understanding why each design choice works, and being able to explain it.